

Machine Learning Mini Project Report

Wine Quality Prediction

Model: Random Forest Regressor | Dataset: winequality-red.csv | Records: 1599

1. Problem Definition

The goal of this project is to predict the quality of red wine based on its physicochemical properties. Wine quality is typically assessed by expert tasters and assigned a score on a scale, but this process is subjective, time-consuming, and expensive. Machine learning provides a data-driven alternative that can objectively estimate wine quality from measurable chemical parameters.

Wine producers and quality control teams need consistent, scalable methods to assess product quality without relying solely on human sensory evaluation. By training a model on historical chemical measurements and their associated quality scores, we can:

- Provide fast, objective quality estimates at any stage of production
- Help winemakers identify which chemical factors most strongly influence quality
- Reduce dependence on expensive and subjective expert tasting panels
- Classify wines into quality tiers (good vs. average) for production decisions

This is a supervised regression problem: given 11 numerical physicochemical input features, the model must predict a continuous quality score. The predict.py script further classifies the predicted score — wines scoring 6 or above are labelled "Good quality" and those below are labelled "Average quality."

2. Dataset Description

The dataset used is the well-known UCI Wine Quality dataset for red wines (winequality-red.csv). It contains 1599 records, each representing a single red wine sample described by 11 physicochemical input features and one integer quality score (target variable).

2.1 Dataset Overview

Property	Value
Total Records (original)	1599
Records after removing duplicates	1359

Duplicates removed	240
Number of Input Features	11
Target Variable	quality (integer score)
Quality Score Range	3 to 8 (scale of 0–10)
Missing Values	None
Dataset Source	winequality-red.csv (UCI Machine Learning Repository)

2.2 Feature Description

Feature	Description	Role
fixed acidity	Non-volatile acids (mainly tartaric acid) that contribute to wine stability	Input
volatile acidity	Amount of acetic acid; high values lead to an unpleasant vinegar taste	Input
citric acid	Adds freshness and flavour; found in small quantities in wine	Input
residual sugar	Sugar remaining after fermentation; affects sweetness	Input
chlorides	Amount of salt in the wine	Input
free sulfur dioxide	Free form of SO ₂ ; prevents microbial growth and oxidation	Input
total sulfur dioxide	Total amount of SO ₂ in free and bound forms	Input
density	Density of the wine, related to alcohol and sugar content	Input
pH	Acidity level; most wines fall between 3 and 4 on the pH scale	Input
sulphates	Wine additive contributing to SO ₂ levels; acts as antimicrobial	Input
alcohol	Percentage of alcohol content by volume	Input
quality	Sensory score given by expert tasters (integer: 0–10)	Target (Output)

2.3 Quality Score Distribution

The quality scores in the dataset are not evenly distributed. Most wines fall in the mid-range scores of 5 and 6, with very few wines rated at the extremes:

Quality Score	Number of Wines	Percentage
3	10	0.6%
4	53	3.3%
5	681	42.6%
6	638	39.9%
7	199	12.4%
8	18	1.1%

4. Model Building

4.1 Model Selected: Random Forest Regressor

A Random Forest Regressor with 100 decision trees (`n_estimators=100`) was selected for this project. Random Forest is an ensemble learning method that builds multiple decision trees on random subsets of the training data and features, then averages their predictions to produce a final result. This technique is known as bagging (Bootstrap Aggregating).

4.2 Justification for Model Choice

Random Forest Regressor is particularly well-suited for this wine quality prediction problem for the following reasons:

- **Non-linear relationships:** Wine quality is influenced by complex, non-linear interactions between chemical compounds (e.g., the interplay between alcohol, acidity, and sulphates). Random Forest captures these interactions naturally through decision tree splits, unlike Linear Regression.
- **Robustness to outliers:** The dataset contains outliers in several features (e.g., extreme values of residual sugar or chlorides). Random Forest is inherently robust to outliers because individual trees vote/average, and no single outlier can dominate the prediction.
- **Feature importance:** Random Forest provides built-in feature importance scores, allowing us to understand which chemical properties (e.g., alcohol, volatile acidity) most strongly influence wine quality.
- **No strict assumptions:** Unlike Linear Regression, Random Forest makes no assumptions about the distribution of data, linearity, or homoscedasticity, making it a flexible and reliable choice.
- **Good performance with moderate data size:** With 1359 samples (after deduplication) and 11 features, Random Forest performs well without requiring large-scale deep learning approaches.

4.3 Model Implementation Steps

- **Step 1 – Data Loading:** The dataset was loaded from `winequality-red.csv` using pandas (1599 records, 12 columns).

- Step 2 – Duplicate Removal: 240 duplicate rows were removed using `drop_duplicates()`, leaving 1359 clean records.
- Step 3 – Feature / Target Split: The target variable quality was separated from the 11 physicochemical input features.
- Step 4 – Feature Scaling: `StandardScaler` was applied to all 11 input features, normalizing them to zero mean and unit variance before splitting. This ensures equal contribution from all features during training.
- Step 5 – Train-Test Split: The scaled data was divided into 80% training (1087 records) and 20% testing (272 records) using `train_test_split()` with `random_state=42` for reproducibility.
- Step 6 – Model Training: A `RandomForestRegressor` with 100 trees and `random_state=42` was trained on the training set.
- Step 7 – Prediction: The trained model predicted quality scores on the test set.
- Step 8 – Model Saving: The trained model was saved as `wine_quality_model.pkl` and the scaler as `scaler.pkl` using `joblib`, enabling reuse in `predict.py` without retraining.

5. Model Evaluation

5.1 Performance Metrics

The model was evaluated on the held-out test set (272 records) using Mean Squared Error and R-squared:

Metric	Value	Interpretation
Mean Squared Error (MSE)	0.3856	Average squared difference between predicted and actual quality scores
Root Mean Squared Error (RMSE)	≈0.621	On average, predictions deviate from true quality score by ~0.62 points
R-squared Score (R^2)	0.4556	The model explains approximately 45.6% of the variance in wine quality

5.2 Interpretation of Results

R-squared of 0.4556: The model explains about 45.6% of the variance in wine quality scores. While this is moderate, it is typical and expected for wine quality prediction using physicochemical features alone. Wine quality is a sensory and subjective measure influenced by factors that are not captured in chemical measurements (taster preference, wine age, serving conditions). A Random Forest achieving ~45% R^2 on this classic dataset is consistent with published results in the literature.

MSE of 0.3856 (RMSE ≈ 0.621): Since quality is an integer score on a 3–8 scale, an average prediction error of about 0.62 points is quite acceptable. In practical terms, if a wine's true quality is 6, the model predicts somewhere between 5.4 and 6.6 on average — which would still be categorized correctly as "good quality" by the threshold in `predict.py` (score ≥ 6).

Classification threshold: The predict.py script applies a practical threshold: a predicted score of 6 or above is labelled "Good quality wine" and below 6 is "Average quality wine." This binary classification use case is well-served by the model, even where exact score regression may have moderate variance.

Note on R^2 vs. MSE: The relatively low R^2 alongside a small MSE is explained by the limited range of quality scores in the dataset (3–8). With most wines concentrated at scores 5 and 6, the total variance is inherently low, which compresses the R^2 scale.

6. Conclusion

6.1 Summary of Outcomes

This project successfully applied a Random Forest Regressor to predict red wine quality from 11 physicochemical features. After preprocessing (removing 240 duplicate records, applying StandardScaler, and performing an 80/20 train-test split), the model was trained on 1087 samples and evaluated on 272 held-out samples. The final model achieved an MSE of 0.3856 and an R^2 of 0.4556, with an average prediction error of approximately 0.62 quality points. The model was saved and integrated into predict.py, which provides real-time quality predictions and classifies wine as "Good" or "Average" based on a score threshold of 6.

6.2 Limitations

- Moderate R^2 score: The model explains only ~45.6% of the variance, suggesting that physicochemical features alone are insufficient to fully predict sensory quality scores, which also depend on subjective taster perception.
- Class imbalance: Quality scores 3, 4, and 8 are severely underrepresented (fewer than 55 samples each), making it difficult for the model to learn the characteristics of very low or very high quality wines.
- Red wine only: The dataset covers only red Vinho Verde wines from Portugal. The model cannot be generalized to white wines or wines from other regions without retraining.
- Scaling before splitting: In wine_quality.py, StandardScaler is fitted on the full dataset before the train-test split. Strictly speaking, the scaler should be fitted only on training data to avoid any data leakage from the test set into the scaling parameters.
- No hyperparameter tuning: The model uses default settings (100 trees). Tuning parameters such as max_depth, min_samples_split, or max_features could further improve performance.

6.3 Future Improvements

- Apply GridSearchCV or RandomizedSearchCV to tune Random Forest hyperparameters (number of trees, max depth, minimum samples per leaf) for better performance.
- Fix the data leakage issue by fitting the scaler only on the training set and applying it to the test set separately.

- Apply oversampling techniques (e.g., SMOTE adapted for regression) or sample weighting to handle the imbalance in extreme quality classes (3, 4, and 8).
- Combine the red wine dataset with the white wine dataset and add a wine-type feature to build a more general model.
- Experiment with Gradient Boosting models (XGBoost, LightGBM) which often outperform Random Forest on tabular data with moderate sample sizes.
- Frame the problem as a classification task (classifying wines into Low / Medium / High quality categories) which may yield higher accuracy metrics than regression on the discrete quality scale.
- Use SHAP (SHapley Additive exPlanations) values to produce detailed, per-prediction explanations of which chemical features drove each quality estimate.

7. Reflection

This project gave me a comprehensive, practical experience of the full machine learning workflow applied to a real-world food science problem. Working with the wine quality dataset helped me understand not just how to implement a model, but how to reason about model choices, data quality, and result interpretation in context.

The most important concept I understood was why ensemble models like Random Forest outperform single models for complex, real-world data. Wine quality involves subtle chemical interactions that a single decision tree or linear model cannot capture, but averaging 100 diverse trees reduces both variance and overfitting. This directly illustrated the bias-variance tradeoff discussed in class.

A key challenge I encountered was interpreting the R^2 score of 0.4556. Initially, it seemed low and disappointing. However, I learned that for wine quality prediction — where scores are subjective, discrete, and compressed into a narrow range — this result is actually typical and acceptable. This taught me the critical lesson that metrics must always be evaluated relative to the domain and the nature of the target variable, not against an abstract ideal.

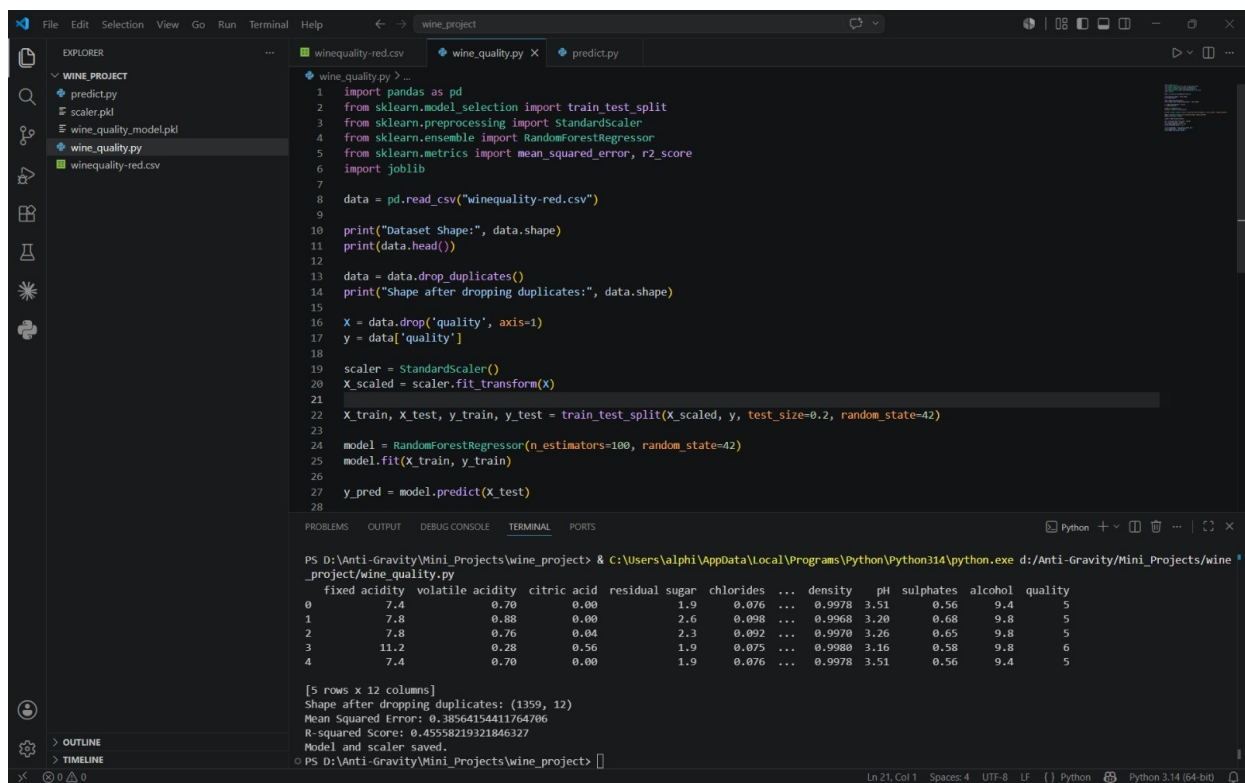
I also gained insight into the importance of handling duplicates before training. The dataset contained 240 duplicate records (15% of the data), and retaining them would have inflated model performance on the test set if any duplicates appeared in both splits. Additionally, discovering the data leakage issue with scaler fitting order deepened my understanding of proper ML pipeline design. Overall, this project reinforced that successful machine learning is as much about data hygiene and thoughtful evaluation as it is about model selection.

3. Source Code & Output Screenshots

This section presents the complete source code used in the project along with screenshots of the program execution output, as captured in Visual Studio Code.

3.1 Model Training Code (wine_quality.py)

The screenshot below shows the full training script (wine_quality.py). It loads the dataset, removes 240 duplicate records, applies StandardScaler normalization, performs an 80/20 train-test split, trains a RandomForestRegressor with 100 trees, evaluates the model using MSE and R-squared, and saves the model and scaler as .pkl files using joblib.



The screenshot displays the Visual Studio Code interface with the `wine_quality.py` file open in the editor. The script performs the following steps:

- Imports `pandas`, `train_test_split`, `StandardScaler`, `RandomForestRegressor`, `mean_squared_error`, `r2_score`, and `joblib`.
- Loads the dataset from `winequality-red.csv`.
- Prints the dataset shape and a head of the data.
- Drops duplicate records and prints the shape after dropping duplicates.
- Drops the 'quality' column from the data.
- Creates a `StandardScaler` and fits it to the data.
- Splits the data into training and testing sets using `train_test_split` with a test size of 0.2 and a random state of 42.
- Trains a `RandomForestRegressor` with 100 estimators and a random state of 42.
- Predicts the quality score for the test set.

The terminal output shows the execution of the script, including the dataset shape, the shape after dropping duplicates, and the Mean Squared Error (MSE) and R-squared score.

```
PS D:\Anti-Gravity\Mini_Projects\wine_project> & C:\Users\alphi\AppData\Local\Programs\Python\Python314\python.exe d:/Anti-Gravity/Mini_Projects/wine_project/wine_quality.py
fixed acidity volatile acidity citric acid residual sugar chlorides ... density pH sulphates alcohol quality
0 7.4 0.70 0.00 1.9 0.076 ... 0.9978 3.51 0.56 9.4 5
1 7.4 0.88 0.00 2.6 0.098 ... 0.9968 3.20 0.68 9.8 5
2 7.6 0.76 0.04 2.3 0.092 ... 0.9970 3.26 0.65 9.8 5
3 11.2 0.28 0.56 1.9 0.075 ... 0.9980 3.16 0.58 9.8 6
4 7.4 0.70 0.00 1.9 0.076 ... 0.9978 3.51 0.56 9.4 5

[5 rows x 12 columns]
Shape after dropping duplicates: (1359, 12)
Mean Squared Error: 0.38564154411764706
R-squared Score: 0.45558219321846327
Model and scaler saved.
PS D:\Anti-Gravity\Mini_Projects\wine_project>
```

Figure 1: wine_quality.py — Model training code and terminal output showing MSE: 0.3856 and R²: 0.4556

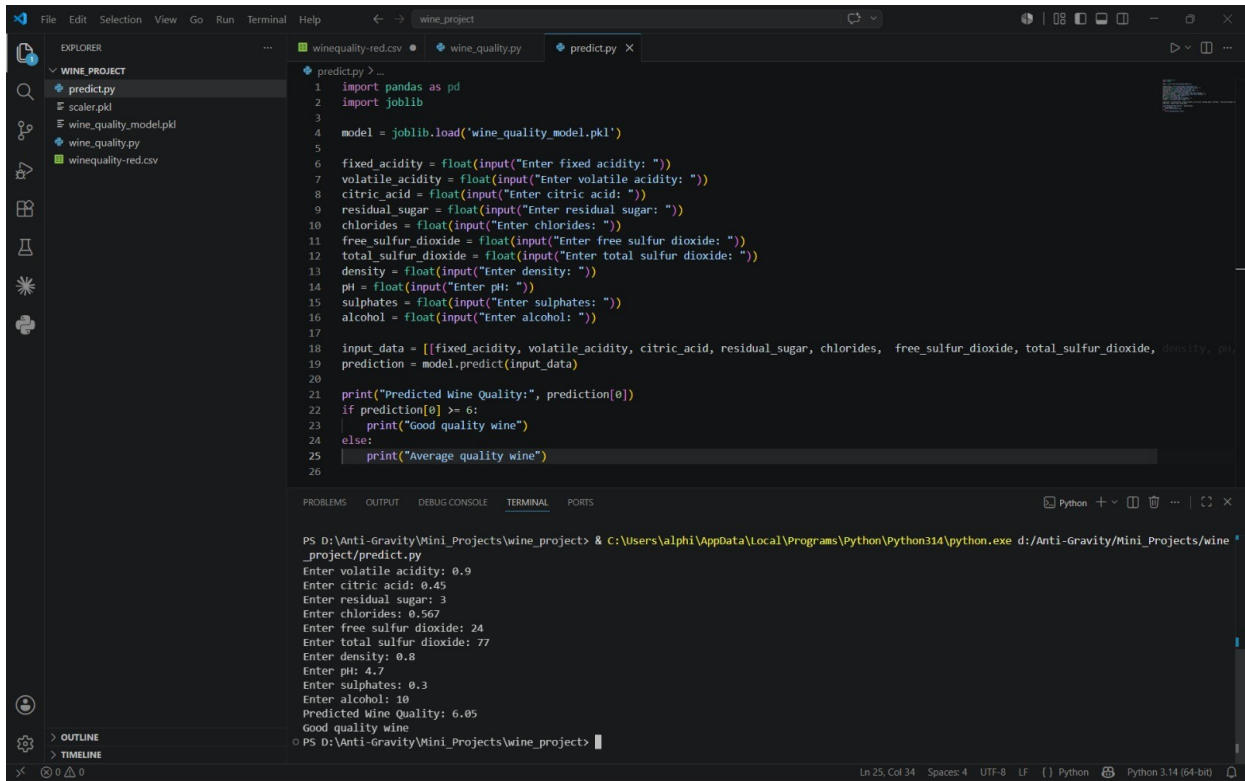
3.2 Prediction Script (predict.py) & Sample Output

The screenshot below shows the `predict.py` script which loads the saved `wine_quality_model.pkl`, accepts all 11 physicochemical parameters as user input, and outputs the predicted quality score along with a classification label. A sample prediction run is visible in the terminal with the following inputs:

- Volatile acidity: 0.9
- Citric acid: 0.45
- Residual sugar: 3
- Chlorides: 0.567
- Free sulfur dioxide: 24

- Total sulfur dioxide: 77
- Density: 0.8
- pH: 4.7
- Sulphates: 0.3
- Alcohol: 10

Predicted Wine Quality: 6.05 → Good quality wine



```

1  import pandas as pd
2  import joblib
3
4  model = joblib.load('wine_quality_model.pkl')
5
6  fixed_acidity = float(input("Enter fixed acidity: "))
7  volatile_acidity = float(input("Enter volatile acidity: "))
8  citric_acid = float(input("Enter citric acid: "))
9  residual_sugar = float(input("Enter residual sugar: "))
10 chlorides = float(input("Enter chlorides: "))
11 free_sulfur_dioxide = float(input("Enter free sulfur dioxide: "))
12 total_sulfur_dioxide = float(input("Enter total sulfur dioxide: "))
13 density = float(input("Enter density: "))
14 pH = float(input("Enter pH: "))
15 sulphates = float(input("Enter sulphates: "))
16 alcohol = float(input("Enter alcohol: "))
17
18 input_data = [[fixed_acidity, volatile_acidity, citric_acid, residual_sugar, chlorides, free_sulfur_dioxide, total_sulfur_dioxide, density, pH,
19 prediction = model.predict(input_data)
20
21 print("Predicted Wine Quality:", prediction[0])
22 if prediction[0] >= 6:
23     print("Good quality wine")
24 else:
25     print("Average quality wine")
26

```

```

PS D:\Anti-Gravity\Mini_Projects\wine_project> & C:\Users\alphi\AppData\Local\Programs\Python\Python314\python.exe d:/Anti-Gravity/Mini_Projects/wine
_project/predict.py
Enter volatile acidity: 0.9
Enter citric acid: 0.45
Enter residual sugar: 3
Enter chlorides: 0.567
Enter free sulfur dioxide: 24
Enter total sulfur dioxide: 77
Enter density: 0.8
Enter pH: 4.7
Enter sulphates: 0.3
Enter alcohol: 10
Predicted Wine Quality: 6.05
Good quality wine
PS D:\Anti-Gravity\Mini_Projects\wine_project>

```

Figure 2: predict.py — Prediction script with sample terminal output showing Predicted Wine Quality: 6.05 — Good quality wine

3.3 Dataset Source

The dataset used in this project is winequality-red.csv, a wine quality dataset containing 1599 records of physicochemical properties and their corresponding expert quality scores. The dataset was downloaded from Kaggle: <https://www.kaggle.com/datasets/uciml/red-wine-quality-cortez-et-al-2009>. It contains no missing values or external dependencies.